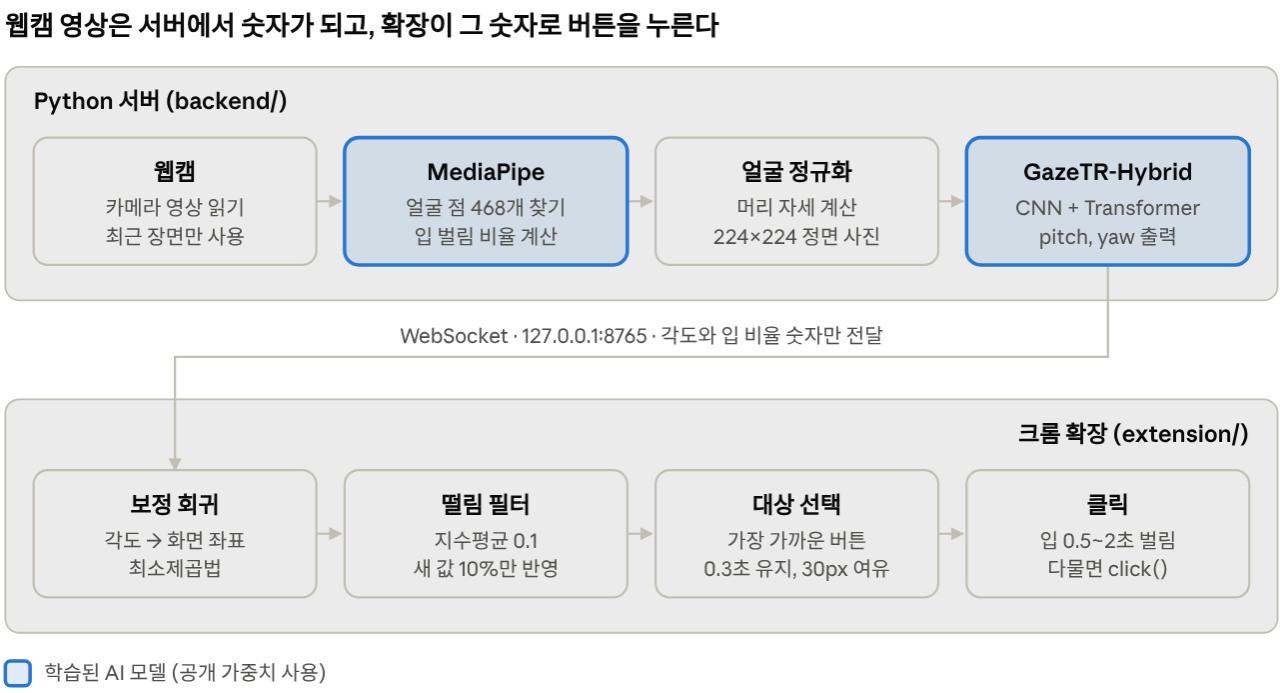
시선 클릭 프로그램 원리 정리 (발표 질문 대비)

Oct 23, 2026 · @miy

전체 흐름 한눈에

프로그램은 **Python 서버(영상 → 숫자)** 와 **크롬 확장(숫자 → 클릭)** 두 부분으로 나뉘고, 둘은 WebSocket으로 연결된다.



프로그램 구조 · 서버 4단계, 확장 4단계

윗줄은 웹캠 사진 한 장이 시선 각도가 되는 과정(1~3장), 가운데 화살표는 통신(4장), 아랫줄은 그 각도로 버튼을 골라 누르는 과정(5~7장)이다.

1. 웹캠과 얼굴 점 (MediaPipe)

웹캠 사진 한 장(프레임)마다 MediaPipe Face Mesh가 얼굴 위 **468개 점**의 위치(x, y 픽셀 좌표)를 찾는다. MediaPipe는 구글이 공개한 AI 모델이고, 우리는 그 결과인 좌표만 쓴다.

- **최신 프레임만 쓰는 이유**: 카메라는 별도 스레드가 계속 읽고(`backend/camera.py` 의 LatestCamera), 처리할 때는 가장 최근 사진 하나만 꺼낸다. 밀린 사진을 차례로 처리하면 커서가 몇 초 전의 시선을 따라가게 된다.
- **카메라 여는 방식**: Windows는 DirectShow, Linux는 V4L2라는 운영체제별 방식으로 연다. 운영체제 차이는 이 파일과 설치·실행 스크립트에만 있다.
- **468개 점 중 실제로 쓰는 점**

  - 눈꼬리 4개 + 코 양옆 2개 → 머리 자세 계산 (2장)
  - 입술 안쪽 위·아래 2개 + 입꼬리 2개 → 입 벌림 비율 (7장)
  - 양쪽 눈 주변 6개씩 → 눈 감김 비율 (눈 감기 클릭 모드용)
- **속도**: 눈·입 상태는 매 프레임 계산한다. 시선 계산(GazeTR)은 CPU에서 한 번에 약 0.11초가 걸려 다른 스레드에서 따로 돌린다. 그래서 시선 값은 **초당 약 9번** 들어온다. 이 숫자는 6장의 필터 지연 계산에 쓰인다.

2. 얼굴 정규화와 224×224

정규화는 웹캠 사진을 **"카메라가 얼굴 정면 60cm 앞에서 찍은 것처럼"** 다시 그리는 과정이다. GazeTR은 이렇게 맞춘 사진으로 학습했기 때문에, 같은 모양으로 넣어야 제대로 답한다. (코드: `backend/tracker.py`의 `face_warp`)

1. **머리 자세 계산 (solvePnP)**: 평균적인 사람 얼굴의 3D 모양(ETH-XGaze 얼굴 모델, mm 단위)에서 눈꼬리 4점과 코 양옆 2점의 3D 위치를 알고 있다. 사진 속 같은 6점의 2D 위치와 맞춰 보면, 얼굴이 카메라 기준으로 얼마나 돌아갔고(회전) 얼마나 떨어져 있는지(이동)를 거꾸로 계산할 수 있다. PnP는 Perspective-n-Point(n개 점으로 자세 구하기)의 줄임말이다.
2. **가상 카메라 돌리기**: 얼굴 중심(두 눈 중심과 코 중심의 가운데)을 정면으로 보도록 가상 카메라를 돌리고, 얼굴의 좌우축이 수평이 되게 맞춘다.
3. **거리 맞추기**: 얼굴까지 거리가 600mm가 되도록 확대·축소한다 (배율 = 600 ÷ 실제 거리).
4. **원근 변환 (warpPerspective)**: 1~3을 3×3 행렬 하나로 합친 뒤, 원본 사진의 픽셀을 옮겨 224×224 사진을 새로 만든다. 행렬은 $가상카메라 \times 배율 \times 회전 \times (실제카메라)^{-1}$ 순서로 곱한다. 뒤에서부터 읽으면 "픽셀 → 실제 방향 → 얼굴 정면 기준으로 회전 → 거리 맞춤 → 새 사진의 픽셀"이다.

하필 224인 이유

- **모델이 그 크기로 학습됐다.** ETH-XGaze 데이터셋과 GazeTR은 224×224 얼굴 사진을 쓴다. 다른 크기를 넣으면 눈이 학습 때와 다른 위치·크기에 있어 답이 틀어진다.
- **구조적으로도 맞아야 한다.** 앞부분 CNN(ResNet-18)은 사진을 가로·세로 절반씩 5번 줄여 $2^5 = 32$분의 1로 만든다. 224 ÷ 32 = 7이라 7×7 = 49칸이 되고, 뒷부분 Transformer는 정확히 49칸 + 1칸 = 50개의 위치 번호를 갖고 있다. 256을 넣으면 8×8 = 64칸이 되어 개수가 맞지 않는다.
- **관례와 속도.** 224는 ImageNet 사진 분류 모델(ResNet 등) 이후 표준 크기다. 더 작으면 눈동자 세부가 뭉개지고, 더 크면 계산량이 넓이에 비례해 늘어 CPU에서 느려진다.

한계: 카메라 렌즈의 실제 초점거리를 재지 않고 "사진 가로 폭"으로 추정한다. 그래서 얼굴 거리 계산에 오차가 있고, 이 오차는 뒤의 보정 단계가 일부 흡수한다.

3. GazeTR-Hybrid: CNN + Transformer

GazeTR-Hybrid는 224×224 얼굴 사진을 받아 **시선 방향 각도 2개 (pitch, yaw)** 를 내놓는 신경망이다. 앞부분은 CNN, 뒷부분은 Transformer라서 'Hybrid(혼합)'라고 부른다. 저자가 공개한 학습 가중치를 그대로 쓰며, 우리가 다시 학습시키지는 않았다.

CNN (합성곱 신경망) — "무늬 찾기"

- 작은 필터(예: 3×3 숫자표)를 사진 위로 한 칸씩 밀면서, 각 위치의 픽셀과 곱해 더한다. 값이 크면 그 자리에 필터와 비슷한 무늬(가로선, 둥근 가장자리 등)가 있다는 뜻이다.
- 층을 쌓을수록 선 → 눈꺼풀 모양 → 눈동자 위치처럼 더 복잡한 특징을 잡는다. 필터의 숫자는 사람이 정하는 게 아니라 학습으로 정해진다.
- 이 모델에서는 ResNet-18이 224×224×3(색) 사진을 **7×7칸 × 칸마다 32개 숫자**로 요약한다. 얼굴을 49칸으로 나눠 칸마다 "여기에 어떤 무늬가 있는지"를 적은 것이다.

Transformer — "칸끼리 서로 참고하기"

- 49칸 + 정답을 모을 빈칸 1개(cls 토큰) = 50개에 각각 위치 정보를 더한다.
- **self-attention**: 각 칸이 나머지 모든 칸을 보고 "나와 관련 있는 칸"에 더 큰 가중치를 주어 정보를 섞는다. 예를 들어 왼눈 칸이 오른눈 칸과 얼굴 기울기를 함께 참고한다. CNN은 가까운 픽셀끼리만 보지만, Transformer는 멀리 떨어진 부분의 관계를 한 번에 본다.
- 이 과정을 6층 반복하고(한 층에 관점 8개), 마지막에 cls 칸의 32개 숫자를 일차식 계산(Linear)으로 2개로 줄여 답을 낸다.

시선 각도의 종류

이름	뜻	비유
pitch	위·아래 각도	고개를 끄덕이는 방향
yaw	왼쪽·오른쪽 각도	고개를 도리도리하는 방향
roll (쓰지 않음)	기울기	고개를 갸우뚱하는 방향

- 시선은 눈에서 나가는 "방향(화살표)"이라 각도 2개면 정해진다. 지구 위 위치가 위도·경도 2개로 정해지는 것과 같다. 화살표를 제자리에서 돌리는 roll은 방향을 바꾸지 않는다.
- 단위는 라디안(1라디안 ≈ 57.3°)이고, (0, 0)은 정규화된 가상 카메라를 정면으로 보는 방향이다. 예: 발표 사진에서는 pitch −32.1°, yaw −21.2°가 나왔다.
- 이 각도는 "카메라 기준 방향"일 뿐 화면의 어느 픽셀인지는 모른다. 그래서 5장의 보정이 필요하다.

학습 데이터와 출처: ETH-XGaze 데이터셋(약 110명을 여러 각도에서 찍은 얼굴 사진에 실제 시선 각도를 붙인 자료)로 학습되었다. 논문은 Cheng & Lu, "Gaze Estimation using Transformer" (ICPR 2022)이고, 코드·가중치는 비영리 조건(CC BY-NC-SA 4.0)으로 공개되어 있다 (`vendor/NOTICE.md`).

4. WebSocket: 서버와 크롬 확장의 통신

WebSocket은 한 번 연결해 두면 서버와 브라우저가 **서로 아무 때나 메시지를 보낼 수 있는** 통신 방식이다. 시선·입 상태가 초당 수십 번 오가야 해서 썼다.

- **일반 웹(HTTP)과의 차이**: HTTP는 브라우저가 요청해야 서버가 답하는 "편지" 방식이다. 새 값이 있는지 매번 물어봐야 해서 느리고 낭비가 크다. WebSocket은 "전화 통화"처럼 연결을 열어 두고, 서버가 새 값이 생기는 즉시 보낸다.
- **연결 구조**: Python 서버(FastAPI + uvicorn)가 `127.0.0.1:8765`의 `/ws` 주소를 연다. 크롬 확장의 백그라운드 스크립트(`worker.js`)가 `ws://127.0.0.1:8765/ws`로 접속하고, 받은 값을 지금 보는 탭의 `content.js`에 전달한다. 보정·필터·선택·클릭은 `content.js`가 `core.js`의 함수를 불러 처리한다.
- **127.0.0.1의 뜻**: '내 컴퓨터 자신'을 가리키는 주소다. 다른 컴퓨터에서는 접속할 수 없다. 웹캠 영상은 서버 밖으로 나가지 않고, 확장에는 숫자만 간다. 예: `{"type":"frame", "gaze":[-0.56, -0.37], "mouthOpen":false, "mar":0.04}` (JSON 형식)
- **막는 장치 3가지**
  1. 접속한 쪽이 크롬 확장(`chrome-extension://`)이 아니면 끊는다. 일반 웹사이트가 몰래 접속하는 것을 막는다.
  2. 연결 후 5초 안에 **연결 토큰**(서버 시작 때 콘솔에 보이는 암호, `runtime.json`에 저장)을 보내야 하고, 틀리면 끊는다.
  3. 한 번에 한 연결만 카메라를 쓸 수 있다.
- **왜 둘로 나눴나**: AI 모델(PyTorch, MediaPipe)은 Python에서 가장 쉽게 돌고, 웹페이지의 버튼을 누르는 일은 브라우저 확장만 할 수 있다. WebSocket이 둘을 잇는다.

5. 보정: 회귀와 최소제곱법

보정은 "시선 각도 2개 → 화면 좌표 2개"로 바꾸는 식을 **사용자마다 새로 만드는** 과정이고, 그 식의 숫자(계수)는 최소제곱법으로 구한다. (코드: `extension/core.js`의 `fit`, `regression`, `solve`)

회귀란: 입력과 출력의 짝 여러 개를 보고 둘 사이의 관계식을 찾는 방법이다. 키와 몸무게 점들에 가장 잘 맞는 직선을 긋는 것이 대표적인 예다. 여기서 입력은 (yaw, pitch), 출력은 화면 x(또는 y)다.

자료 모으기: 화면 가로·세로 4%, 35%, 65%, 96% 위치의 4×4 = 16점을 차례로 보게 한다. 점마다 0.9초 기다렸다가 시선 값을 12개 이상 모으고, **중앙값**을 그 점의 대표값으로 쓴다. 깜빡임 같은 튀는 값은 평균은 끌어당기지만 중앙값은 거의 바꾸지 못하기 때문이다.

식이 왜 이 모양인가

$$x = a_0 + a_1 \cdot yaw + a_2 \cdot pitch \qquad y = b_0 + b_1 \cdot yaw + b_2 \cdot pitch$$

- 고개를 고정하면 시선 각도가 조금 변할 때 화면 위 점도 거의 비례해서 움직인다. 화면 위치는 거리 × tan(각도)인데, 작은 각도에서는 tan(각도) ≈ 각도이기 때문이다. 그래서 1차식으로 충분히 가깝다.

- a_0는 기준점(카메라·앉은 위치), a_1은 좌우 각도가 1 변할 때 x가 얼마나 변하는지(화면까지 거리), a_2는 위아래 각도가 x에 섞이는 정도(카메라나 얼굴의 기울기)다.
- 실제 코드는 각도에서 평균을 빼고 표준편차로 나눈 뒤(표준화) 계산한다. 숫자 크기를 비슷하게 맞춰 계산 오차를 줄이려는 것이고, 식의 모양은 같다.

최소제곱법: 오차의 제곱을 더한 값을 가장 작게

16점 각각에서 실제 좌표와 식의 예측값 차이(오차)를 구한다. 오차를 그냥 더하면 +와 −가 상쇄되므로 제곱해서 더한다.

$S = \Sigma\,(x - a_0 - a_1 \cdot yaw - a_2 \cdot pitch)^2$ (16개 점에 대해 모두 더함)

S는 a에 대한 2차함수(아래로 볼록한 그릇 모양)라서, a_0, a_1, a_2 각각으로 미분한 값이 0인 곳이 최솟값이다. 그러면 미지수 3개짜리 연립방정식(정규방정식)이 나오고, 행렬로 쓰면 다음과 같다. X는 한 줄이 [1, yaw_i, $pitch_i$]인 16×3 표다.

$(X^TX)\cdot a = X^Tx$ (X^T는 X의 행과 열을 바꾼 행렬)

코드에서 `regression`이 X^TX(3×3)와 X^Tx를 만들고, `solve`가 가우스-조던 소거법으로 푼다. 중학교 연립방정식의 가감법을 순서대로 반복하는 방법이다. x용 한 번, y용 한 번 푼다.

손으로 해 보는 예 (입력 1개): 점 (1, 2), (2, 3), (3, 5)에 y = a + bx를 맞춘다. $\Sigma x = 6$, $\Sigma x^2 = 14$, $\Sigma y = 10$, $\Sigma xy = 23$이므로 정규방정식은 3a + 6b = 10, 6a + 14b = 23이다. 풀면 b = 1.5, a = 1/3이고, 이 직선이 세 점의 오차 제곱합을 가장 작게 만든다. 보정은 이것을 입력 2개짜리로 한 것이다.

2차식과 교차검증 (과적합 막기)

- 2차식은 yaw^2, yaw·pitch, $pitch^2$ 항을 더한 계수 6개짜리 식이다. 16점에는 더 잘 맞지만, 새 위치에서는 크게 틀릴 수 있다. 문제집 답을 외운 학생이 새 문제를 못 푸는 것과 같아서 이를 **과적합**이라 한다.
- **하나 빼기 교차검증**: 16점 중 1점을 빼고 15점으로 식을 구해 뺀 점을 예측한다. 이것을 16번 반복해 "처음 보는 점에서의 오차"를 잔다.
- 2차식의 교차검증 오차가 1차식보다 **15% 이상** 작을 때만 2차식을 쓴다. 2차식에는 계수가 너무 커지지 않도록 S에 벌점(계수²의 합 × 0.01, 0.1, 1 중 가장 나은 것)을 더한다.

검증 5점: 학습에 쓰지 않은 5점(네 모서리 근처와 가운데)으로 실제 오차를 잔다. 기준보다 크면 먼저 "식 전체를 일정하게 한쪽으로 밀기"를 한 번 시도하고 다시 검증한다. 그래도 크면 평균·최대 오차를 보여 주고 "그래도 사용 / 다시 보정"을 고르게 한다. 시연 영상의 "검증 평균 181px · 최대 295px"가 이 경우다.

6. 떨림 필터: 이동평균과 지수평균

필터는 흔들리는 시선 좌표 여러 개를 섞어 부드럽게 만든다. 기본값은 **지수평균 0.1**이고, F 키로 끄기 / 이동평균 10·20 / 지수평균 0.2·0.1을 바꿀 수 있다. 대상 선택에는 필터를 거친 좌표를 쓴다.

이동평균: 최근 N개 좌표의 평균이다. 떨림이 무작위라면 서로 상쇄되어, 떨림 크기(표준편차)가 약 $1/\sqrt{N}$로 줄어든다. N = 10이면 약 1/3, N = 20이면 약 1/4.5다.

지수평균: 새 측정값 m과 바로 전 결과 s를 일정한 비율로 섞는다. 이 프로그램은 α = 0.1이다.

$s(t) = \alpha \cdot m(t) + (1 - \alpha) \cdot s(t-1) = 0.1 \cdot m(t) + 0.9 \cdot s(t-1)$

왜 0.1과 0.9를 곱하나

- **두 수의 합이 1이어야 "평균"이 된다.** 합이 1이면 두 값 사이의 한 점(가중평균)이 된다. 합이 1이 아니면 가만히 봐도 값이 변한다. 예를 들어 0.5와 0.6(합 1.1)을 쓰고 같은 점 100을 계속 넣으면 110 → 116 → 119.6…처럼 100에서 벗어난다.
- **α는 "새 값을 얼마나 믿을지"다.** α = 0.1이면 새 측정은 10%만 반영하고, 지금까지의 결과를 90% 유지한다.
- **펼쳐 쓰면 이름의 뜻이 보인다.** s_t를 계속 대입하면 아래처럼 과거 값의 가중치가 0.9배씩 줄어드는 평균이 된다. 가중치가 지수함수처럼 줄어들어 "지수"평균이고, 가중치를 모두 더하면 등비급수 0.1 ÷ (1 − 0.9) = 1이다.

$s(t) = 0.1 \cdot m(t) + 0.09 \cdot m(t-1) + 0.081 \cdot m(t-2) + 0.0729 \cdot m(t-3) + \dots$

- **저장할 것이 1개뿐이다.** 이동평균은 최근 N개를 모두 기억해야 하지만, 지수평균은 바로 전 결과 하나만 있으면 된다.

대가: 부드러울수록 늦다

시선을 한쪽에서 다른 쪽으로 한 번에 옮겼을 때, 커서가 절반까지 따라오는 데 걸리는 시간을 코드로 재보았다 (시선 값은 초당 약 9번).

필터	절반 따라오는 데 필요한 값 개수	약 몇 초
이동평균 10	5	0.6초
지수평균 0.2	4	0.4초
이동평균 20	10	1.1초
지수평균 0.1 (기본값)	7	0.8초

처음에는 이동평균 5·지수평균 0.45를 썼지만 떨림이 많이 남았다. 실제로 써 보니 0.8초 정도 늦는 것보다 떨리는 것이 더 불편해서 강한 필터를 기본값으로 했다. 패널의 그래프에서 빨간 점은 필터 전, 파란 점은 필터 후다.

7. 대상 선택과 입 벌림 클릭

시선 좌표는 버튼 하나를 정확히 가리킬 만큼 정밀하지 않다. 그래서 커서를 정확히 맞추는 대신 **시선에서 가장 가까운 버튼을 고르고**, 입을 벌렸다 다물면 그 버튼을 누른다. (코드: `core.js`의 `NearestSelector`, `MouthClick`, `backend/gestures.py`)

대상 선택

- 시선 근처의 누를 수 있는 요소(버튼, 링크 등)를 모아 거리순으로 줄 세운다. 거리는 시선 점에서 버튼 사각형까지의 최단 거리이고, 사각형 안이면 0이다.
- **머무름 0.3초**: 같은 버튼이 0.3초 동안 계속 1등이어야 선택한다. 시선이 잠깐 스친 버튼은 무시된다.
- **끈적임 30px**: 이미 선택된 버튼이 있으면, 새 1등이 그보다 30px 이상 더 가까워야 바꾼다. 두 버튼 경계에서 선택이 왔다 갔다 하는 것을 막는다.
- 이처럼 "바꾸는 기준"과 "되돌리는 기준"을 다르게 두는 것을 **히스테리시스**라고 한다. 에어컨이 25°C에서 켜지고 23°C에서 꺼져서, 24°C 근처에서 켜졌다 꺼졌다를 반복하지 않는 것과 같다.
- 선택된 버튼에는 초록 테두리가 생긴다.

입 벌림 클릭

MAR(입 벌림 비율) = 입술 안쪽 위·아래 거리 ÷ 입 너비(양쪽 입꼬리 거리)

- **비율을 쓰는 이유**: 거리(픽셀)만 쓰면 카메라에서 멀어질 때 작아진다. 두 거리가 같은 비율로 작아지므로 나눈 값은 그대로다.
- **두 기준 (여기도 히스테리시스)**: 0.30보다 크면 "열림", 0.20보다 작으면 "닫힘", 그 사이면 이전 상태를 유지한다. 기준이 하나면 값이 기준 근처에서 흔들릴 때 열림·닫힘이 빠르게 반복된다.
- **대상 고정**: 입이 열리는 순간의 선택 버튼을 기억해 둔다. 입을 벌리며 얼굴이 움직여 시선이 흔들려도 누를 버튼은 바뀌지 않는다.
- **시간 조건**: 0.5초 이상 2초 이하로 벌렸다가 다물 때 클릭한다. 0.5초보다 짧으면(말할 때) 무시하고, 2초보다 길면 취소한다. 클릭 뒤 0.8초는 다시 클릭하지 않고, 얼굴을 놓치거나 값이 0.4초 넘게 끊기면 진행 중인 클릭을 취소한다.
- **실제 클릭 방법**: 웹페이지 요소의 `focus()`와 `click()`을 부른다. 마우스 커서는 움직이지 않는다. 그래서 웹페이지 안의 버튼·링크는 누를 수 있지만, 브라우저 메뉴나 다른 프로그램은 누를 수 없다.
- 눈 감기 클릭(0.6~2초 양눈 감기)도 `CLICK_MODE = 'eyes'`로 남아 있다. 눈을 감는 동안에는 시선을 잴 수 없어 시선 계산을 잠시 멈춘다.

예상 질문과 답

- **Q. 정확도는 얼마나 되나?** — 시연 때 검증 5점의 평균 오차가 약 181px(최대 295px)였다. 화면 가로의 약 10분의 1이다. 그래서 커서를 정확히 맞추지 않고 "가장 가까운 버튼 고르기" 방식을 썼다.
- **Q. AI는 어디에 썼나?** — 얼굴 점 찾기(MediaPipe)와 시선 각도 계산(GazeTR) 두 곳이다. 보정·필터·버튼 선택·클릭 판단은 회귀, 평균, 거리 비교 같은 수학 규칙으로 동작한다.
- **Q. 회귀도 AI 아닌가?** — 넓게 보면 가장 단순한 머신러닝이다. 다만 신경망이 아니라 계수 3개짜리 식을 사용자 한 명의 자료 16개로 맞추는 것이다.
- **Q. GazeTR을 학습시켰나?** — 아니다. 논문 저자가 ETH-XGaze로 학습해 공개한 가중치를 그대로 썼다. 대신 사람마다 다른 부분은 보정으로 맞춘다.
- **Q. 왜 보정점이 16개인가?** — 2차식은 계수가 6개라 점이 넉넉해야 교차검증이 의미가 있다. 또 화면 4%~96%까지 덮어야 가장자리에서도 식이 맞는다. 코드는 16점이 모여야 2차식을 고려한다.

- **Q. 고개를 움직이면?** — 정규화가 머리 회전을 일부 되돌리지만, 보정식은 보정한 자리 기준이다. 자리를 크게 옮기면 패널의 "다시 보정"을 누른다.
- **Q. 왜 마우스 커서를 움직이지 않나?** — 오차가 커서 점 하나로는 너무 크다. 크롬 확장은 웹페이지 버튼들의 위치를 알아서 "가장 가까운 버튼"을 고를 수 있고, 운영체제의 마우스 제어 권한도 필요 없다.
- **Q. 반응이 얼마나 늦나?** — 대략 1초 안팎이다. 시선 계산 약 0.11초, 필터가 절반 따라오는 데 약 0.8초, 버튼 머무름 0.3초가 겹쳐 있어서 정확히 더하기는 어렵다.
- **Q. 왜 GPU가 아니라 CPU인가?** — 학교 PC에 GPU가 없어도 돌아가게 하려고 CPU용 PyTorch만 설치한다. 모델 한 번 계산에 약 0.11초(스레드 2개 측정)이다.
- **Q. 개인정보는 안전한가?** — 웹캠 영상은 내 컴퓨터의 Python 서버 밖으로 나가지 않는다. 확장에는 127.0.0.1로 숫자만 가고, 토큰이 맞아야 연결된다. 기본 설정에서는 인터넷으로 아무것도 보내지 않는다.
- **Q. 안경을 쓰면?** — 따로 시험해 보지 않았다. 모르는 것은 "확인하지 못했다"고 답하는 것이 가장 안전하다.